DATA 442/642 · Advanced Machine Learning · Week 5

5.3 Support Vector Machines

Margins, the dual, and the kernel trick.

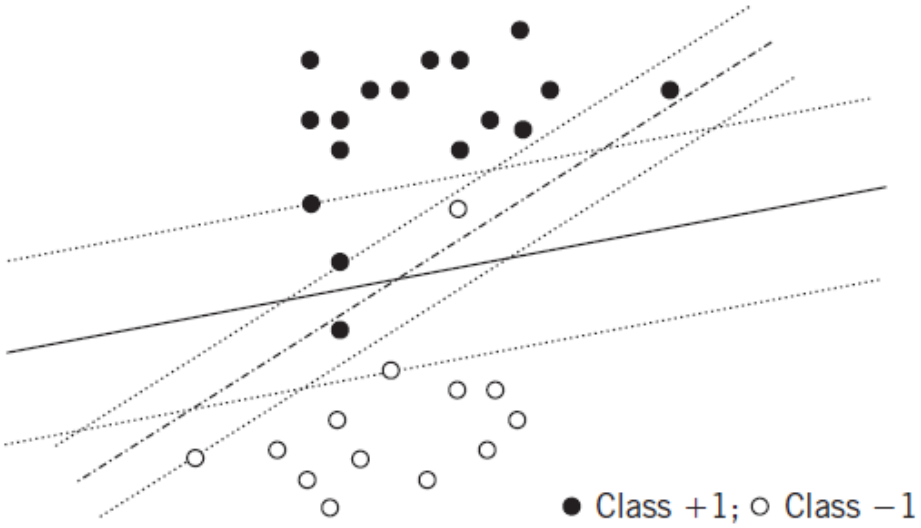
These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

The figures follow Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*, Theodoridis, and Géron, *Hands-On Machine Learning*.

1. The main idea

Think of open and filled circles as houses in two villages. A road between them should be as wide as possible and should demolish as few houses as possible. No sensible engineer picks the dash-dotted path in the figure.

A classifier should sit between the dense regions of the two classes, in a sparse belt, leaving the **largest possible margin**. That is the usual requirement for **generalization**: small error on points that were not in the training set.



---

2. Linearly separable classes

If the classes are linearly separable, infinitely many hyperplanes classify the training set with zero error. From that family one can always keep those that satisfy

$$y_n(\theta^\top x_n + \theta_0) \geq 1, \qquad n = 1, \ldots, N.$$

The SVM picks the one with smallest norm:

$$\underset{\theta \in \mathbb{R}^I}{\text{minimize}} \quad \frac{\|\theta\|^2}{2} \quad \text{subject to} \quad y_n(\theta^\top x_n + \theta_0) \geq 1, \quad n = 1, \ \ldots, N.$$

$\|\theta\|$ is tied directly to the **margin**.

---

3. Primal, dual, and support vectors

The constrained problem is the **primal**. It has a **dual**. In general the dual only lower-bounds the primal; under convexity (and regularity of the constraints) the values match. SVM meets those conditions, so you may solve either.

The Lagrangian is

$$\mathcal{L}(\theta, \theta_0, \lambda) = \frac{1}{2}\|\theta\|^2 - \sum_{n=1}^{N} \lambda_n \left(y_n(\theta^\top x_n + \theta_0) - 1 \right).$$

KKT stationarity gives

$$\nabla_\theta \mathcal{L} = 0 \quad \Rightarrow \quad \hat{\theta} = \sum_{n=1}^{N} \lambda_n y_n x_n.$$

and

$$\nabla_{\theta_0} \mathcal{L} = 0 \quad \Rightarrow \quad \sum_{n=1}^{N} \lambda_n y_n = 0, \quad \lambda_n \left(y_n(\theta^\top x_n + \theta_0) - 1 \right) = 0, \quad \lambda_n \geq 0.$$

Plug that expansion into $\mathcal{L}$ and solve the dual with a **quadratic program**:

$$\underset{\lambda}{\text{minimize}} \quad \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m x_n^\top x_m - \sum_{n=1}^{N} \lambda_n \quad \text{subject to} \quad \lambda_n \geq 0, \quad \sum_{n=1}^{N} \lambda_n y_n = 0.$$

With a dual solution $\hat{\lambda}$,

$$\hat{\theta} = \sum_{n=1}^{N_s} \lambda_n y_n x_n,$$

where N_s is the number of **nonzero** multipliers. Only points that meet the inequality as an equality, $y_n(\theta^\top x_n + \theta_0) = 1$, have $\lambda_n \neq 0$. Those points are the **support vectors**. Recover $\hat{\theta}_0$ from any such equality.

---

4. Notes

- The SVM solution is **unique**: the cost is strictly convex.

- The **dual** is usually faster than the primal.
- The dual is what makes the **kernel trick** possible. In an RKHS the predictor is

$$\hat{y}(x) = \sum_{n=1}^{N_s} \lambda_n y_n \kappa(x, x_n) + \theta_0 \,.$$

---

5. Non-separable classes

When the classes overlap, allow slack:

$$\underset{\theta \in \mathbb{R}^l}{\text{minimize}} \quad \frac{\|\theta\|^2}{2} + C \sum_{n=1}^{N} \xi_n \quad \text{subject to} \quad y_n(\theta^\top x_n + \theta_0) \geq 1 - \xi_n, \quad \xi_n \geq 0 \,.$$

A **margin error** is $y_n(\theta^\top x_n + \theta_0) < 1$, i.e. $\xi_n > 0$. If $\xi_n = 0$ that point does not add to the cost. The optimizer tries to drive as many ξ_n as possible to zero.

The user parameter C trades the two terms. Large C: a **small** margin, fewer margin errors. Small C: the opposite.

---

6. Choice of hyper-parameters

C is almost always chosen by **cross-validation** on held-out data.

The other choice is the **kernel**. Different kernels, different performance. A Gaussian kernel needs enough training points to fill the input space, because $\kappa(x, x_n)$ matches x to the stored point x_n. Training-set size matters.

Kernel	Formula	Typical use
Linear	$\kappa(x, x') = x^\top x'$	Linearly separable data, or many features relative to N
Polynomial	$\kappa(x, x') = (x^\top x' + c)^d$	Polynomial-shaped boundaries (e.g. some image tasks)
Gaussian RBF	$\kappa(x, x') = \exp\left(-\|x - x'\|^2 / (2\sigma^2) \right)$	Complex nonlinear relations
Sigmoid	$\kappa(x, x') = \tanh(\alpha x^\top x' + c)$	Periodic or sigmoid-like relations (e.g. some time series)

---

7. Practice

1. Where do the support vectors appear in the SVM dual?
2. What does a large C do to the margin and to margin errors?
3. Why does the dual make the kernel trick possible?